

4주차

📅 날짜	2026년 9월 29일
☰ 다중 선택	
≡ 텍스트	파머완 4.5 ~ 4.8, 4.10~4.11

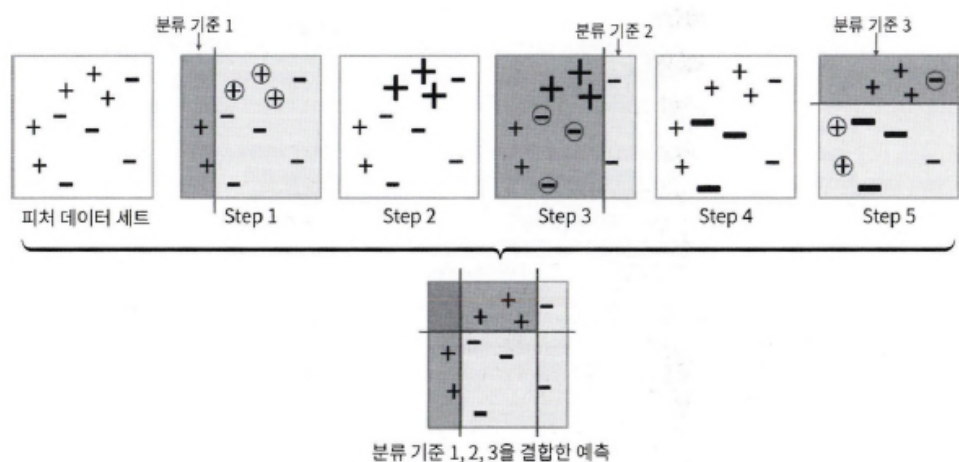
4. 분류

5. GBM(Gradient Boosting Machine)

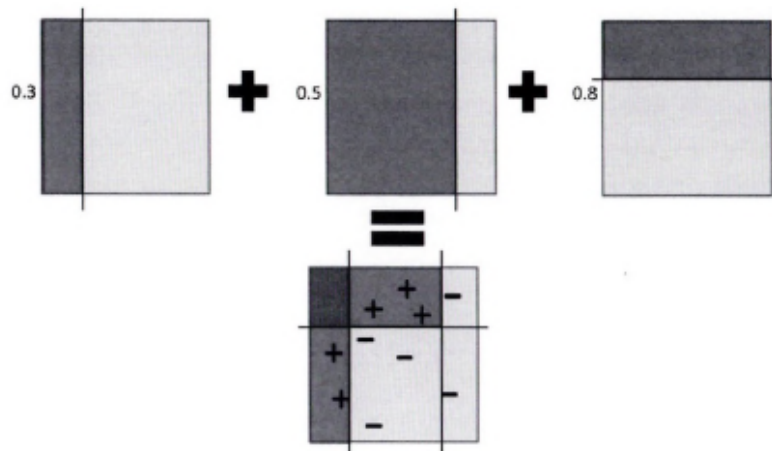
1) GBM의 개요 및 실습

부스팅

- 여러 개의 약한 학습기(weak learner) 순차적으로 학습-예측하면서 잘못 예측한 데이터에 가중치 부여 → 오류 개선해가며 학습
- 부스팅 대표적인 구현
 - AdaBoost(Adaptive boosting) :
오류 데이터에 가중치 부여하면서 부스팅 수행



개별 약한 학습기는 각각 가중치를 부여해 결합



◦ 그래디언트 부스트 (GBM) :

가중치 업데이트 시 **경사 하강법(Gradient Descent)**을 이용

경사하강법 - 반복 수행을 통해 오류를 최소화할 수 있도록 가중치의 업데이트 값을 도출하는 기법

사이킷런은 GradientBoostingClassifier 클래스 제공해서 GBM 기반 분류 수행
일반적으로 랜덤 포레스트 보다 예측 성능 조금 뛰어남.

but, 수행시간 오래 걸림 & 하이퍼 파라미터 튜닝 노력 필요

2) GBM 하이퍼 파라미터 소개

- loss : 경사 하강법에서 사용할 비용함수 지정. 기본값 'deviance'
- learning_rate : GBM이 학습 진행할 때마다 적용하는 학습률. 기본값 0.1
- n_estimators : weak learner 개수. 개수 많을 수록 예측성능
- subsample : weak learner 가 학습에 사용하는 데이터 샘플링 비율. 기본값 1(전체 학습 데이터 기반으로 학습)

GBM은 과적합에도 강한 뛰어난 예측 성능 가진 알고리즘. but, 수행시간 오래 걸림

6. XGBoost(eXtra Gradient Boost)

XGBoost 개요

- 트리 기반의 앙상블 학습에서 가장 각광받고 있는 알고리즘
- GBM에 기반하지만, GBM의 단점인 느린 수행 시간&과적합 규제 부재 등의 문제 해결 가능
- 특히 병렬 CPU 환경에서 병렬 학습 가능 - > GBM 보다 빠르게 학습 완료

주요 일반 파라미터

- `booster` : `gbtree`(tree based model) 또는 `gblinear`(linear model) 선택. 디폴트는 `gbtree`.
- `silent` : 디폴트는 0이며, 출력 메시지를 나타내고 싶지 않을 경우 1로 설정
- `nthread` : CPU의 실행 스레드 개수를 조정, 디폴트는 CPU 전체 스레드 다 사용

주요 부스터 파라미터

- `eta` : GBM의 학습률
- `num_boost_rounds` : GBM의 `n_estimators`와 같은 파라미터
- `min_child_weight` : 트리에서 추가적으로 가지를 나눌지를 결정하기 위해 필요한 데이터들의 `weight` 총합.
- `gamma` : 트리의 리프 노드를 추가적으로 나눌지를 결정할 최소 손실 감소 값
- `max_depth` : 트리 기반 알고리즘의 `max_depth`와 동일
- `sub_sample` : GBM의 `subsample`과 동일
- `colsample_bytree` : GBM의 `max_features`와 유사
- `lambda` : L2 Regularization 적용 값
- `alpha` : L1 Regularization 적용 값
- `scale_pos_weight` : 특정 값으로 치우친 비대칭한 클래스로 구성된 데이터 세트의 균형을 유지하기 위한 파라미터

학습 태스크 파라미터

- **objective**: 최소화할 손실 함수 지정
- `binary:logistic` : 이진 분류 / 예측 확률 반환
- `multi:softmax` : 다중 분류 / 클래스 번호 반환 / `num_class` 필요
- `multi:softprob` : 다중 분류 / 클래스별 예측 확률 반환 / `num_class` 필요

7. LightGBM

- **LightGBM**: XGBoost와 대표적인 부스팅 알고리즘
- 주요 장점

- 빠른 학습·예측 속도
- 적은 메모리 사용량
- 대용량 데이터 처리에 유리
- 병렬 컴퓨팅, GPU 지원
- 카테고리형 피처 자동 처리
- 예측 성능: XGBoost와 큰 차이 없음
- 단점
 - 적은 데이터에서 과적합 발생 가능성
 - 일반적으로 10,000건 이하 데이터에서 주의
- 트리 분할 방식
 - XGBoost 등: **Level Wise** → 균형 있게 분할
 - LightGBM: **Leaf Wise** → 손실이 가장 큰 리프를 우선 분할
 - → 학습 속도 및 예측 오류 최소화에 유리
- 패키지: `lightgbm`
- 사이킷런 래퍼
 - `LGBMClassifier` → 분류
 - `LGBMRegressor` → 회귀
- 사용법: `fit()`, `predict()` → 사이킷런과 동일

LightGBM 하이퍼파라미터 튜닝 핵심

- `num_leaves`: 모델 복잡도 조절의 핵심 파라미터
 - ↑ → 정확도 ↑ 가능 / 복잡도·과적합 ↑
- `min_child_samples`: 과적합 방지
 - ↑ → 트리 깊이 ↓ / 과적합 ↓
- `max_depth`: 트리 깊이 제한
 - `num_leaves`, `min_child_samples` 와 함께 조절

기본 튜닝 방법

- `learning_rate` ↓ + `n_estimators` ↑

- 부스팅의 기본적인 튜닝 방법
- `n_estimators` 가 지나치게 크면 과적합 주의

과적합 방지

- `reg_lambda` / `reg_alpha` → 규제 적용
- `colsample_bytree` → 사용할 피쳐 비율 조절
- `subsample` → 사용할 데이터 비율 조절

8. 베이지안 최적화 기반 HyperOpt를 이용한 하이퍼 파라미터 튜닝

베이지안 최적화 개요

베이지안 최적화

- 정의: 목적 함수 식을 모르는 **블랙박스 함수**에서 적은 시도로 최적 입력값 탐색
- 목적: 최소한의 탐색으로 최적 하이퍼 파라미터 발견
- 장점: Grid Search보다 **효율적인 탐색**

주요 구성 요소

- **대체 모델 (Surrogate Model)**
 - 관측된 데이터를 기반으로 **목적 함수 예측**
 - 예: Gaussian Process, TPE
- **획득 함수 (Acquisition Function)**
 - 대체 모델을 기반으로 **다음에 탐색할 입력값 결정**
 - → 다음 하이퍼 파라미터 추천

베이지안 최적화 과정

1. 하이퍼 파라미터 랜덤 샘플링
2. 모델 학습 및 성능 측정
3. 대체 모델 업데이트
4. 획득 함수로 다음 하이퍼 파라미터 선택
5. 반복 → 최적 하이퍼 파라미터 탐색

HyperOpt

- **HyperOpt**: 베이지안 최적화 기반 하이퍼 파라미터 튜닝 라이브러리
- `fmin()`: 목적 함수의 **최솟값** 탐색
- **검색 공간**: 하이퍼 파라미터의 탐색 범위 설정
- **목적 함수**: 하이퍼 파라미터 입력 → 모델 학습 → 성능 반환

! HyperOpt 유의사항

- `fmin()` → **최솟값**을 찾음
 - 정확도처럼 **클수록 좋은 지표** → `accuracy` 로 반환
- 정수형 파라미터
 - HyperOpt 검색값은 실수형이므로 `int()` **형변환 필요**
 - 예: `max_depth=int(params['max_depth'])`

10. 분류 실습 - 캐글 신용카드 사기 검출

언더 샘플링 & 오버 샘플링 핵심 정리

불균형 데이터 문제

- **불균형 데이터**: 레이블별 데이터 개수 차이가 큰 데이터
- **정상 데이터** ↑ / **이상 데이터** ↓
- 문제: 정상 데이터에 치우친 학습 → **이상 데이터 탐지 어려움**

언더 샘플링 (Undersampling)

- 다수 클래스의 데이터를 감소
- 예: 정상 10,000개 → 100개
- 장점: 데이터 불균형 완화 / 학습 시간 감소
- 단점: 정상 데이터가 너무 많이 제거되어 **정보 손실**
- → 다수 클래스 데이터를 소수 클래스 수준으로 줄임

오버 샘플링 (Oversampling)

- 소수 클래스 데이터를 증가

- 장점: 소수 클래스의 학습 데이터 확보 / 정보 손실 적음
- 단점: 단순 복제 → 과적합 발생 가능
- → 일반적으로 언더 샘플링보다 예측 성능 측면에서 유리한 경우가 많음

SMOTE

- 대표적인 오버 샘플링 기법
- 소수 클래스 데이터의 K-최근접 이웃(KNN) 활용
- 기존 데이터와 조금씩 다른 새로운 데이터 생성
- 단순 데이터 복제 → ✗
- 새로운 합성 데이터 생성 → ○
- Python 패키지: `imbalanced-learn`

11. 스택킹 앙상블

스태킹(Stacking) 핵심 정리

- 스택킹: 여러 개의 모델을 결합하여 최종 예측 성능 향상
- 핵심: 개별 모델의 예측 결과를 다시 학습 데이터로 사용
- 배깅·부스팅과 차이: 개별 모델의 예측 결과를 이용해 새로운 모델이 최종 예측

구성

- 기반 모델(Base Model)
 - 여러 개의 개별 ML 모델
 - 각각 학습 → 예측값 생성
- 메타 모델(Meta Model)
 - 기반 모델들의 예측값을 입력 데이터로 학습
 - 최종 예측 수행

동작 과정

원본 데이터 → 여러 기반 모델 학습 → 각 모델의 예측값 생성 → 예측값을 결합(Stacking)
→ 메타 모델 학습 → 최종 예측

특징

- 여러 모델의 **예측 결과를 새로운 피처로 활용**
- 반드시 성능 향상이 보장되지는 않음
- 일반적으로 **성능이 비슷한 여러 모델을 결합**
- 많은 개별 모델을 활용할수록 효과를 기대할 수 있음
- 캐글 등 성능 경쟁에서 활용

CV 기반 스택킹

Step 1. 기반 모델의 메타 데이터 생성

- K-Fold(예: 3-Fold)로 학습 데이터 분할
- 각 Fold마다:
 - **2개 Fold → 기반 모델 학습**
 - **1개 Fold → 검증 및 예측**
- 3번 반복하여 **검증 폴드의 예측값 저장**
- 저장된 예측값 → **메타 모델의 학습 데이터**

테스트 데이터 처리

- 각 Fold의 학습 데이터로 기반 모델 학습
- **원본 테스트 데이터 예측**
- 3번의 예측 결과를 **평균**
- 평균 예측값 → **메타 모델의 테스트 데이터**

Step 2. 메타 모델 학습

- 기반 모델들의 **예측값을 결합**
- → 메타 모델의 **학습 데이터**
- 원본 테스트 데이터에 대한 기반 모델의 **평균 예측값**
- → 메타 모델의 **테스트 데이터**
- 메타 모델 학습 → **최종 예측**